

Database Clients

Maulana Hafidz Ismail

06 Nov 2025

Contents

1	Interacting with a MySQL database using Python	1
1.1	Database Clients	1
1.2	Bagaimana Databases digunakan dalam Programming	2
1.3	Desain Database	3
1.4	Model Data Inti	3
1.5	Pertimbangan Utama	3
1.6	Tantangan Modifikasi Database	3
1.7	Kepercayaan Pengguna	4
1.8	Koneksi antara Python dan MySQL	4
1.9	Proses Koneksi	4
1.10	Menggunakan Cursor untuk Eksekusi Query	4
1.11	Penutupan Koneksi dan Contoh Sintaks Utuh	4
1.12	Rangkuman terkait pip	5
1.12.1	Ulasan Singkat pip	5
1.12.2	Fungsionalitas pip	5
1.13	Menghubungkan Python dengan MySQL	7
1.13.1	Penjelasan Kode	8
1.14	Membuat Alias untuk MySQL Connector	10
1.15	Membuat Koneksi ke Database	10
1.16	Koneksi dan Cursor	10
1.17	Sintaks untuk Membuat Database	11
1.18	Menggunakan Database	11
1.19	Membuat Tabel	11
1.20	Pengertian Cursor	11
1.21	Karakteristik Cursors	12
1.22	Cara Menggunakan Cursor	12
1.22.1	Contoh Penggunaan	13
1.23	Pengertian Kelas Cursor	13
1.24	Subkelas Cursor	13
1.24.1	Contoh Penggunaan	13
1.25	Sintaks untuk Menginstansiassi Cursor	13
2	Performing Queries in MySQL using Python	15
2.1	Membuat Data (Create Data)	15
2.2	Membaca Data (Read Data)	15
2.3	Mengambil Hasil (Fetching Results)	15
2.4	Penutupan Koneksi	15
2.5	commit() Setelah Transaksi	16
2.6	Pembaruan Data (Updating Records)	16
2.7	Penghapusan Data (Deleting Records)	16
2.8	Teknik Filtering	17
2.9	Teknik Sorting	17
2.10	Pengertian JOIN	17
2.11	Jenis-jenis JOIN	17

2.12	Proses Menggunakan INNER JOIN dengan Python	18
3	Advanced Database Clients	19
3.1	Pengantar Fungsi MySQL	19
3.2	Jenis-jenis Fungsi di MySQL	19
3.3	Penggunaan Fungsi dengan Python	19
3.4	Pengantar Fungsi Datetime	20
3.5	Fungsi Utama dalam Datetime	20
3.6	Contoh Sintaks untuk Mengupdate Slot Pemesanan	20
3.7	Pengantar Tentang Datetime	21
3.8	Fungsi-fungsi Dalam Datetime	21
3.9	Menggunakan Datetime untuk Mengatur Slot Peemesanan	21
3.10	Apa itu Stored Procedures	22
3.11	Langkah-langkah Menggunakan Stored Procedures	22
3.12	Manfaat Stored Procedures	22
3.13	Sintaks untuk Membuat Stored Procedure	22
3.14	Mengakses Stored Procedure Menggunakan Python	22
3.14.1	Memanggil dan Mengambil Hasil dari Stored Procedures .	23
3.15	Apa itu Database Connection Pooling?	24
3.16	Cara Kerja Connection Pooling	24
3.17	Mengatasi Keterbatasan Koneksi	24
3.18	Keuntungan Connection Pooling	24
3.19	Database Connexion Pool	24
3.20	MySQL Connection Pool Module	25
3.21	Metode Kelas dalam MySQL Connection Pool	25
3.22	Langkah-langkah Membuat Connection Pool	25
4	Working with a Database Client	27

1 Interacting with a MySQL database using Python

1.1 Database Clients

MySQL adalah sistem manajemen basis data SQL sumber terbuka. Ini adalah basis data relasional yang menyimpan data dalam tabel-tabel terpisah. Perangkat lunak basis data MySQL mengikuti sistem client/server, dan server dapat dijalankan di desktop atau laptop tanpa memerlukan pengawasan yang signifikan. MySQL cepat, andal, skalabel, dan mudah digunakan.

Sistem klien/server MySQL terdiri dari server SQL yang mendukung berbagai program klien dan antarmuka pemrograman aplikasi (API) untuk mengelola basis data.

Dalam sistem client/server, MySQL sebagai server basis data adalah perangkat lunak yang berfungsi sebagai backend tempat Anda dapat membuat dan menyimpan basis data. Sedangkan client adalah perangkat lunak yang dapat Anda gunakan untuk terhubung ke server basis data MySQL guna melakukan operasi yang diperlukan menggunakan kueri SQL pada basis data. Client memungkinkan Anda untuk mengintegrasikan basis data MySQL ke dalam aplikasi Anda.

Database MySQL dirancang untuk bekerja dengan berbagai perangkat lunak, program, dan bahasa pemrograman. Semua perangkat lunak tersebut memiliki klien khusus untuk berinteraksi dengan database, dan klien-klien ini juga disebut driver atau API.

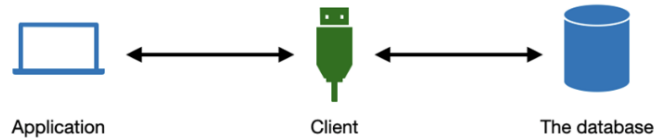


Figure 1:

Dalam diagram, klien adalah sepotong kode yang menghubungkan aplikasi front-end Anda dengan basis data back-end. Klien membangun koneksi komunikasi antara aplikasi dan basis data untuk menjalankan kueri SQL sesuai dengan kebutuhan aplikasi Anda.

Saat bekerja dengan Python, terdapat berbagai klien yang tersedia. Dua klien yang paling sering digunakan adalah **MySQL Connector/Python** dan **SQLAlchemy**. Klien MySQL Connector/Python sangat populer dan merupakan klien atau driver database standar untuk platform dan pengembangan Python. Klien SQLAlchemy juga memberikan pengembang Python kendali penuh dan fleksibilitas untuk menjalankan kueri SQL menggunakan Python pada database MySQL.

Untuk terhubung ke database menggunakan Python, Anda perlu mengirimkan permintaan ke klien untuk membuat koneksi dengan database. Setelah permintaan Anda diterima oleh database, database akan mengirimkan pesan konfirmasi ke klien. Python kemudian menggunakan koneksi tersebut untuk menjalankan kueri SQL. Dengan demikian, klien memungkinkan Anda untuk bekerja dengan database MySQL di belakang layar dari aplikasi Python Anda.

1.2 Bagaimana Databases digunakan dalam Programming

Berikut adalah role pekerjaan yang terkait dengan database:

- Database administrator

Seorang administrator basis data adalah penjaga data. Mereka merancang pendekatan untuk mengoptimalkan pembacaan dan penulisan informasi. Sebagai pemecah masalah, peran ini melibatkan kerja sama dengan berbagai departemen dalam suatu organisasi.

Pengetahuan tentang SQL merupakan aset besar bagi administrator karena memungkinkan mereka membuat kueri terlepas dari mesin basis data yang digunakan.

Peran ini memerlukan pengalaman bekerja di posisi entry-level seperti analis data atau programmer. Salah satu fitur berguna dari sintaks SQL adalah bahwa setelah Anda dapat menulis pernyataan SQL di satu mesin, Anda dapat menulisnya di mesin mana pun.

- Database modeler

Peran seorang perancang model basis data adalah untuk merancang skema basis data. Mereka bertugas untuk memelihara dokumentasi mengenai arsitektur basis data serta solusi informasi yang digunakan. Seorang perancang model yang baik harus memahami jenis-jenis data dalam basis data, kapan menggunakan jenis data yang sesuai, dan bagaimana menormalisasi tabel untuk memastikan kueri dan gabungan data yang baik di masa mendatang.

- Database engineer

Seorang insinyur basis data bekerja di bagian backend dan bertugas untuk membuat dan mengisi basis data. Seorang programmer yang baik menggunakan fungsi dan prosedur. Mereka juga sering memiliki bahasa pemrograman tambahan yang dapat digunakan untuk berinteraksi dengan basis data, seperti Python atau R. Hal ini karena data tidak datang secara terpisah. Informasi memiliki sumber asal, seperti situs web, dan tujuan akhir, seperti lokasi penyimpanan online.

Sebagai programmer basis data, tugas Anda adalah menulis kode yang dapat mengakses dan memperbarui data ini dengan efisien dan lancar. Seorang insinyur juga dapat bertindak sebagai analis data. Insinyur basis data juga dapat menjadi peran spesialis.

- Database analyst

Peran analis adalah memanfaatkan informasi yang terdapat dalam basis data untuk keperluan bisnis. Untuk memahami data tersebut, diperlukan pemahaman yang baik tentang cara mengakses tabel-tabel yang berbeda secara efisien dan mengembalikan data dalam format yang dapat diproses dengan mudah dalam operasi selanjutnya.

- Database tester

Seorang penguji basis data bertugas untuk menemukan bug dalam sistem. Ini adalah peran khusus yang membutuhkan pemikiran kreatif dalam mengidentifikasi kelemahan yang mungkin ada dalam sistem. Peran ini memerlukan seseorang yang mandiri dan nyaman bekerja di luar tim.

1.3 Desain Database

- Data Validity: Memastikan bahwa data yang disimpan dan ditampilkan akurat dan dapat dipercaya.
- Data Integrity: Merujuk pada akurasi dan konsistensi data dalam database, yang didukung oleh desain yang baik untuk mencegah akses yang tidak sah.

1.4 Model Data Inti

- Core Data Model: Langkah awal dalam desain database yang melibatkan pembuatan model data inti yang diperlukan agar produk berfungsi. Ini membantu dalam memahami cara mengakses data dari perspektif produk.
- Privacy and Validation: Setelah model data awal dibangun, pertimbangan untuk privasi dan validasi, seperti enkripsi data, sangat penting.

1.5 Pertimbangan Utama

- User Privacy: Pengguna harus diberitahu tentang di mana data mereka disimpan dan bagaimana data tersebut digunakan untuk menjaga kepercayaan.
- Scalability: Database harus mampu tumbuh seiring dengan meningkatnya jumlah pengguna dan produk, memerlukan pandangan jauh ke depan dalam desain.

1.6 Tantangan Modifikasi Database

Mengubah model data yang ada bisa menjadi tantangan dan memerlukan perencanaan yang cermat untuk migrasi data.

1.7 Kepercayaan Pengguna

Desain dan manajemen database yang baik sangat penting untuk membangun kepercayaan pengguna, memastikan pengguna merasa aman tentang penanganan data mereka.

1.8 Koneksi antara Python dan MySQL

- Koneksi antara aplikasi Python dan database MySQL dibentuk menggunakan **Application Programming Interface (API)**. API adalah sekumpulan program yang berfungsi sebagai jembatan antara aplikasi Python dan database MySQL.
- API yang umum digunakan adalah **MySQL Connector Python**.

1.9 Proses Koneksi

- Proses dimulai dengan aplikasi Python mengirimkan connection request ke API. Connection Request adalah permintaan dari aplikasi Python untuk mengakses dan mengambil informasi dari database.
- Setelah permintaan diterima, API meneruskan permintaan tersebut ke database MySQL, yang kemudian mengonfirmasi bahwa koneksi telah berhasil dibentuk.

1.10 Menggunakan Cursor untuk Eksekusi Query

- Setelah koneksi berhasil, Anda dapat membuat cursor dari kelas connector. Cursor adalah objek yang digunakan untuk mengeksekusi query SQL di database MySQL.
- Contoh penggunaan: Aplikasi Python dapat menggunakan cursor untuk mengecek waktu kedatangan tamu di restoran Little Lemon dengan menjalankan query SQL.

1.11 Penutupan Koneksi dan Contoh Sintaks Utuh

Setelah permintaan selesai, objek cursor dan koneksi database dapat ditutup.
Sintaks Contoh:

```
import mysql.connector

# Membuat koneksi
connection = mysql.connector.connect(
    host="localhost",
    user="username",
    password="password",
    database="database_name"
)
```

```
# Membuat cursor
cursor = connection.cursor()

# Menjalankan query
cursor.execute("SELECT * FROM bookings WHERE guest_id = 1")

# Menutup cursor dan koneksi
cursor.close()
connection.close()
```

1.12 Rangkuman terkait pip

Anda telah mempelajari bahwa Python memerlukan driver untuk terhubung dengan basis data MySQL. Driver adalah perpustakaan yang memungkinkan Anda berinteraksi dengan perangkat lunak lain. Driver berbentuk perpustakaan, yang juga disebut paket.

Beberapa driver tidak terinstal bersama instalasi Python awal Anda, sehingga perlu diimpor. Ini disebut perpustakaan eksternal. Untuk membantu mengelola perpustakaan ini, Anda sebaiknya familiar dengan Pip, penginstal paket bawaan yang disertakan dalam unduhan Python awal Anda. Pip dapat digunakan untuk menginstal, mengonfigurasi, dan memperbarui perpustakaan eksternal.

1.12.1 Ulasan Singkat pip

Pip adalah sistem manajemen paket yang menginstal dan mengelola paket Python di sistem Anda. Paket Python merujuk pada perpustakaan eksternal apa pun yang tidak termasuk dalam unduhan Python awal.

Menggunakan pip untuk menginstal paket Anda adalah cara yang sangat mudah untuk memastikan lingkungan Anda memiliki semua paket yang diperlukan untuk menjalankan aplikasi Anda dengan sukses.

Untuk memeriksa versi pip yang terinstal di lingkungan Anda, buka prompt perintah dan ketik `pip {version}`. Perintah ini akan menunjukkan apakah pip terinstal dan menampilkan versi pip yang berjalan di mesin Anda.

Untuk mengakses pip di lingkungan Jupyter, Anda dapat menjalankan perintah yang sama seperti sebelumnya, tetapi tambahkan tanda seru (!) sebelum pip seperti yang ditunjukkan dalam kode berikut:

```
!pip --version
```

1.12.2 Fungsionalitas pip

Pip memiliki sejumlah perintah berguna yang dapat mempermudah pekerjaan seorang programmer dengan secara otomatis mengonfigurasi dan menginstal program-program tersebut. Mari kita jelajahi beberapa di antaranya.

- Menginstal paket

Menginstal paket menggunakan pip di lingkungan Jupyter sederhana menjalankan perintah pip install paket dengan kode berikut:

```
!pip install mysql-connector
```

- Melihat informasi paket

Anda dapat menggunakan pip untuk mengetahui detail dari suatu paket tertentu dengan menggunakan perintah SHOW sebagai berikut:

```
!pip show mysql-connector
```

Informasi ini dapat membantu Anda memastikan apakah Anda telah menginstal versi yang benar dari suatu perpustakaan. Ketergantungan terjadi ketika suatu perpustakaan memerlukan perpustakaan lain untuk dapat dijalankan.

- Memperbarui perpustakaan

Mengetahui nomor versi perpustakaan dapat membantu memastikan aplikasi Anda berjalan dengan lancar. Perpustakaan terus mengalami perubahan, dan beberapa perubahan dapat mengganggu lingkungan pemrograman Anda.

Jika hal ini terjadi, Anda mungkin perlu memperbarui perpustakaan yang bersangkutan. Anda dapat memperbarui perpustakaan menggunakan pip. Untuk memperbarui perpustakaan di pip, Anda dapat mengetikkan kode berikut:

```
!pip install --upgrade mysql-connector
```

Perintah ini memastikan bahwa versi terbaru dari perpustakaan tersebut tersedia.

- Menampilkan daftar paket yang tersedia di lingkungan kerja

Fungsi berguna lainnya yang disediakan oleh pip adalah kemampuannya untuk menampilkan daftar semua paket yang ada di lingkungan kerja Anda. Untuk mendapatkan daftar cetak semua perpustakaan, gunakan kode berikut:

```
!pip list
```

Ini menampilkan daftar semua paket yang tersedia di lingkungan ini.

- Menampilkan daftar paket yang tersedia di lingkungan pemrograman

Untuk mengekstrak daftar perpustakaan yang ada di lingkungan pemrograman Anda ke berkas teks, gunakan metode freeze dari pip:

```
!pip freeze >requirements.txt
```

Perintah ini pertama-tama memerintahkan pip untuk mengekstrak semua paket yang tersedia. Kurung sudut kemudian mengarahkan pip untuk menyimpan output dalam berkas teks bernama 'requirements.' Anda dapat menentukan lokasi penyimpanan berkas ini dengan menentukan lokasi direktori untuk berkas tersebut, serta memberikan nama kustom yang mudah dikenali.

- Menghapus perpustakaan dari lingkungan kerja
Akhirnya, Anda dapat menggunakan kode berikut jika Anda perlu menghapus perpustakaan dari lingkungan kerja Anda:

```
!pip uninstall mysql-connector
```

Dengan perintah ini, pip akan mencari paket yang ditentukan dan menghapusnya dari sistem Anda.

1.13 Menghubungkan Python dengan MySQL

Anda perlu membuat koneksi antara Python dan MySQL. Contohnya, Little Lemon perlu menghubungkan situs web mereka yang menggunakan Python dengan database MySQL agar pelanggan dapat melihat data seperti menu dan slot pemesanan.

Anda mungkin sudah familiar dengan MySQL Connector Python API, yang memfasilitasi koneksi antara Python dan MySQL. Untuk mengimpornya, ketik `import mysql.connector` di program Python Anda.

Ini adalah perpustakaan eksternal yang berfungsi sebagai driver. Perpustakaan ini mengubah objek string Python menjadi pernyataan SQL yang valid dan dapat dieksekusi di MySQL.

Hasil dari pernyataan SQL tersebut kemudian diparsing dan dikembalikan ke klien Python dalam format yang kompatibel dengan tipe data dan struktur Python.

Untuk terhubung dengan basis data MySQL, program klien Python harus menyediakan parameter koneksi yang sesuai.

Parameter-parameter ini biasanya mencakup spesifikasi informasi berikut:

- host,
- server,
- database tujuan,
- dan nama pengguna serta kata sandi.

Screenshot berikut menampilkan kode untuk koneksi tipikal ke basis data MySQL menggunakan kelas konektor MySQL Python:

```
import mysql.connector as connector

connection = connector.connect(user="mario", password = "cuisine", port = 3306,
                               host = "domain.com", database="little_lemon")
```

Figure 2:

1.13.1 Penjelasan Kode

- Import connector

Baris pertama mengimpor `mysql.connector` sebagai alias yang disebut `connector`. Kode ini mengimpor fungsionalitas `connector` ke dalam lingkungan Python.

- Parameters

Baris kedua menunjukkan bagaimana fungsi `connect` dalam konektor menerima berbagai parameter.

Secara umum, saat terhubung ke database, nama pengguna dan kata sandi harus disertakan sebagai parameter untuk mendapatkan akses ke database MySQL.

Terdapat juga beberapa parameter opsional yang dapat disertakan. Mari kita lihat lebih dekat parameter-parameter ini.

- Port number

Salah satu contoh parameter ini adalah nomor port. Secara default, nomor port untuk basis data MySQL biasanya 3306, tetapi dapat dikonfigurasi ke nomor port lain.

Port adalah cara untuk mengakses komputer. Ini adalah metode untuk memisahkan saluran komunikasi yang masuk dan keluar dari sistem Anda sehingga aplikasi yang berbeda dapat menjalankan kode masing-masing.

Alasan menggunakan port yang berbeda adalah agar aplikasi dapat menerima respons instan saat koneksi yang sesuai dibuat dengan sistem.

Jika hanya ada satu port, maka akan terjadi kemacetan permintaan ke sistem. Hal ini akan membuat tidak mungkin bagi aplikasi untuk mendapatkan respons instan saat permintaan dibuat jika aplikasi lain sedang menggunakan port tersebut.

- Local host

Server lokal digunakan untuk menghosting database. Komunikasi utama dilakukan di dalam laptop itu sendiri.

Namun, dalam organisasi yang lebih besar, pengaturan default ini kemungkinan besar akan dikonfigurasi ke situs hosting eksternal.

Dalam contoh yang diberikan, nama domain contoh `domain.com` digunakan untuk menunjukkan bahwa Anda melakukan panggilan eksternal. Anda dapat mengisi pengaturan ini dengan nama domain atau alamat IP.

- Database name

Akhirnya, Anda juga dapat melewati nama database yang diberikan sebagai parameter. Hal ini akan mengarahkan objek koneksi langsung ke database yang diperlukan. Dalam contoh yang diberikan, objek koneksi diarahkan ke database `little_lemon`.

Namun, dapat ada banyak database yang berbeda, sehingga nama database dapat diubah sesuai kebutuhan.

Sebagai alternatif, Anda juga dapat menggunakan objek kursor dari kelas koneksi untuk mengakses database secara langsung, menggunakan kode berikut:

```
cursor.execute("USE database_name")
```

- Connection issues

Ada banyak masalah potensial yang dapat terjadi saat terhubung ke database. Jika tidak memperhitungkan masalah-masalah ini, aplikasi Anda dapat mengalami crash secara tiba-tiba di lingkungan produksi.

Untuk menghindari masalah-masalah ini, disarankan untuk menggunakan penanganan kesalahan try/except Python, seperti pada contoh berikut:

```
try:
    connection=connector.connect(user="not_mario!", password="cuisine")
except:
    print("there was an error connecting to the database")
there was an error connecting to the database
```

Figure 3:

Try/except adalah blok kode penanganan kesalahan bawaan Python. Blok ini terlebih dahulu menjalankan pernyataan dan memberikan compiler dua kemungkinan untuk dieksplorasi, bukan hanya satu yang mungkin menyebabkan program crash.

Dalam contoh di atas, nama pengguna yang salah menyebabkan blok try gagal. Oleh karena itu, blok except dijalankan.

Mysql-connector memiliki perpustakaan bernama `errorcode` yang dirancang khusus untuk menangani pengecualian ini.

Contoh kode berikut memodifikasi kode contoh di atas sehingga programmer dapat menerima pesan yang akurat mengenai masalah apa yang mungkin menyebabkan kesalahan.

```

import mysql.connector as connector
from mysql.connector import errorcode
try:
    connection=connector.connect(user="mario", password="cuisine",
                                database="big_lemon")
except connector.Error as err:
    if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
        print("connection user or password are incorrect")
    elif err.errno == errorcode.ER_BAD_DB_ERROR:
        print("database does not exist")
    else:
        print(err)

```

database does not exist

Figure 4:

Upaya koneksi awal dapat ditemukan di bagian try. Bagian except berisi serangkaian pernyataan kondisional if/else yang memungkinkan penanganan berbagai masalah koneksi yang mungkin terjadi. Masalah-masalah ini dapat berkaitan dengan koneksi, kredensial, atau berbagai masalah sintaksis.

Pertama, nama pengguna dan kata sandi diperiksa. Kemudian Python memeriksa apakah basis data target ada. Akhirnya, pesan kesalahan terbuka ditampilkan yang mencantumkan referensi ke masalah apa pun yang tidak secara eksplisit dijelaskan dalam kode.

1.14 Membuat Alias untuk MySQL Connector

Menggunakan alias dapat membuat kode Anda lebih efisien. Anda dapat membuat alias dengan sintaks `import mysql.connector as connector`. Ini memungkinkan Anda menggunakan `connector` sebagai pengganti `mysql.connector` di seluruh kode Anda.

Pastikan Anda telah menginstal MySQL Connector API terlebih dahulu, jika tidak, Anda akan mendapatkan kesalahan "module not found".

1.15 Membuat Koneksi ke Database

Untuk membuat koneksi, buat variabel bernama `connection` dan tetapkan nilainya dengan `connector.connect()`. Anda perlu memberikan argumen seperti database, username, dan password.

Dalam contoh ini, username adalah "Mario" dan password adalah "cuisine". Secara default, koneksi dibuat ke database yang diinstal secara lokal, yang disebut "localhost" dan dapat diakses melalui IP 127.0.0.1.

1.16 Koneksi dan Cursor

- Koneksi: Koneksi antara aplikasi Python dan database MySQL telah dibuat menggunakan objek koneksi.

- Cursor: Sebuah objek yang memungkinkan komunikasi dengan database MySQL. Cursor digunakan untuk mengeksekusi query SQL.

1.17 Sintaks untuk Membuat Database

- Untuk membuat database baru, Anda perlu menulis query SQL sebagai string Python dan menyimpannya dalam variabel `create_database_query`. Contoh sintaks:

```
create_database_query = """CREATE DATABASE LittleLemon;
"""
```

- Setelah itu, gunakan metode `execute` dari cursor untuk menjalankan query:

```
cursor.execute(create_database_query)
```

1.18 Menggunakan Database

Setelah database dibuat, Anda perlu mengatur database untuk digunakan dengan query SQL yang disimpan dalam variabel `use_database_query`. Contoh sintaks:

```
use_database_query = """USE LittleLemon;"""
cursor.execute(use_database_query)
```

1.19 Membuat Tabel

Untuk membuat tabel, Anda perlu menulis query SQL untuk tabel yang diinginkan. Misalnya, untuk membuat tabel `menu_items`, Anda bisa menggunakan sintaks berikut:

```
create_menuitem_table = """
CREATE TABLE menu_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200),
    type VARCHAR(100),
    price INT
);
"""
cursor.execute(create_menuitem_table)
```

1.20 Pengertian Cursor

Cursor adalah objek yang digunakan untuk menunjukkan posisi data dalam database. Ini membantu aplikasi Python untuk menemukan data yang diperlukan untuk menyelesaikan query.

1.21 Karakteristik Cursors

- Read-only: Cursors tidak dapat digunakan untuk memperbarui data yang terkait. Data yang diambil tetap tidak dapat diubah.
- Non-scrollable: Cursors mengambil data secara berurutan, sehingga tidak bisa melompat atau mengambil data dalam urutan terbalik.
- Sensitive: Cursors menunjuk ke data asli dalam database, bukan salinan, sehingga lebih cepat dalam pengambilan data.

1.22 Cara Menggunakan Cursor

- Deklarasi Cursor:

Sintaks:

```
DECLARE cursor_name CURSOR FOR SELECT statement;
```

Ini memberi nama pada cursor dan menentukan tujuan penggunaannya.

- Membuka Cursor:

Sintaks:

```
OPEN cursor_name;
```

Ini memulai pencarian hasil dari pernyataan SELECT.

- Mengambil Data:

Sintaks:

```
FETCH cursor_name INTO variable_name;
```

Ini mengambil hasil dan menyimpannya ke dalam variabel di aplikasi Python.

- Menutup Cursor:

Sintaks:

```
CLOSE cursor_name;
```

Menutup cursor adalah praktik baik untuk membebaskan memori yang digunakan.

1.22.1 Contoh Penggunaan

Dalam kasus Little Lemon, mereka dapat menggunakan cursor untuk mengambil data pemesanan tamu.

```
DECLARE guest_booking_details CURSOR FOR SELECT * FROM
    bookings WHERE guest_id = ?;
OPEN guest_booking_details;
FETCH guest_booking_details INTO booking_data;
CLOSE guest_booking_details;
```

1.23 Pengertian Kelas Cursor

Cursor Class adalah kelas yang digunakan untuk menerjemahkan komunikasi antara Python dan database MySQL. Python mengirimkan pernyataan SQL dalam bentuk objek string, dan kelas cursor mengubahnya menjadi perintah yang dapat dipahami oleh MySQL.

1.24 Subkelas Cursor

Kelas-kelas yang diturunkan dari kelas cursor yang memungkinkan pengolahan data dengan cara yang berbeda sesuai kebutuhan.

- Cursor Row Subclass: Mengembalikan hasil tanpa memprosesnya menjadi format yang lebih ramah Python. Ini mengurangi penggunaan daya pemrosesan tetapi memerlukan lebih banyak kode untuk memproses variabel yang ditargetkan.
- MySQL Cursor Dictionary Class: Mengembalikan setiap baris sebagai dictionary, sehingga memudahkan akses variabel dengan menggunakan nama variabel langsung.
- Buffered Cursor Class: Menyimpan subset data dalam memori buffer, sehingga tidak perlu meminta setiap baris dari server berulang kali. Namun, ini hanya dapat digunakan untuk dataset kecil.

1.24.1 Contoh Penggunaan

Interleaving SQL Requests : Proses di mana hasil dari satu kueri digunakan untuk membuat kueri berikutnya. Misalnya, Little Lemon dapat menggunakan ID pemesanan dari kueri pertama untuk menemukan biaya makanan tamu.

1.25 Sintaks untuk Menginstansiassi Cursor

- Untuk membuat cursor standar:

```
cursor = connection.cursor()
```


- Untuk membuat instance dari subclass cursor:

```
cursor = connection.cursor(buffered=True) # untuk
        buffered cursor
cursor = connection.cursor(dictionary=True) # untuk
        dictionary cursor
```

2 Performing Queries in MySQL using Python

2.1 Membuat Data (Create Data)

- Insert Statement: Untuk menambahkan data ke dalam database, seperti menambahkan nama pelanggan dan waktu pemesanan ke dalam tabel bookings. Sintaks:

```
insert_query = "INSERT INTO bookings (customer_name, time) VALUES ('JohnDoe', '18:00')"
```

- Mengonversi ke String Python: Setelah menulis pernyataan SQL, konversikan ke dalam format string Python dengan menambahkan tanda kutip.

2.2 Membaca Data (Read Data)

- Select Statement: Untuk mengambil data dari database, seperti mengambil semua data dari tabel bookings. Sintaks:

```
select_query = "SELECT * FROM bookings"
```

- Eksekusi Query: Setelah menulis pernyataan select, kirimkan ke objek cursor untuk dieksekusi.

2.3 Mengambil Hasil (Fetching Results)

- Fetch All: Setelah mengeksekusi query, gunakan metode fetchall() untuk mengambil semua hasil Sintaks:

```
results = cursor.fetchall()
```

Fetch All adalah metode yang digunakan untuk mengambil semua baris hasil dari query yang telah dieksekusi.

- Mendapatkan Nama Kolom: Untuk mendapatkan nama kolom dari hasil query, buat variabel baru dan ambil nama kolom dari objek cursor.

Nama kolom adalah label yang digunakan untuk mengidentifikasi data dalam tabel.

2.4 Penutupan Koneksi

Setelah selesai, penting untuk menutup objek cursor dan koneksi untuk menghindari kebocoran memori.

Menutup koneksi berarti mengakhiri sesi komunikasi antara aplikasi Python dan database.

2.5 commit() Setelah Transaksi

Fungsi `commit()` dalam konteks database, khususnya saat menggunakan Python dengan MySQL, berfungsi untuk menyimpan semua perubahan yang telah dilakukan dalam transaksi ke database.

Ketika kamu melakukan operasi seperti menambah, mengubah, atau menghapus data dalam database, perubahan tersebut tidak langsung disimpan. Mereka berada dalam "transaksi" yang bisa dibatalkan. Dengan memanggil `commit()`, kamu memberi tahu database bahwa semua perubahan yang telah dilakukan dalam transaksi tersebut harus disimpan. Jika kamu tidak memanggil `commit()`, perubahan tersebut akan hilang ketika koneksi ke database ditutup.

2.6 Pembaruan Data (Updating Records)

- Update Statement: Untuk memperbarui data dalam database, digunakan pernyataan `update`. Contoh sintaks:

```
update_query = "UPDATE bookings SET table_no=10 WHERE booking_id=6"
```

- Eksekusi Query: Setelah menulis query, gunakan metode `execute` untuk menjalankannya. Contoh:

```
cursor.execute(update_query)
```

- Komit Perubahan: Setelah menjalankan query `update`, gunakan metode `commit` untuk menyimpan perubahan. Contoh:

```
connection.commit()
```

`commit` adalah metode yang digunakan untuk menyimpan semua perubahan yang telah dilakukan pada database.

2.7 Penghapusan Data (Deleting Records)

- Delete Statement: Untuk menghapus data dari database, digunakan pernyataan `delete`. Contoh sintaks:

```
delete_query = "DELETE FROM bookings WHERE booking_id=4"
```

- Eksekusi Query: Setelah menulis query, gunakan metode `execute` untuk menjalankannya. Contoh:

```
cursor.execute(delete_query)
```

- Komit Perubahan: Setelah menjalankan query `update`, gunakan metode `commit` untuk menyimpan perubahan. Contoh:

```
connection.commit()
```

2.8 Teknik Filtering

- WHERE Clause: Digunakan untuk memfilter dan mengekstrak record yang memenuhi kondisi tertentu. Contoh sintaks:

```
SELECT booking_id, bill_amount FROM orders WHERE  
    bill_amount = 40;
```

WHERE adalah klausa dalam SQL yang digunakan untuk menentukan kondisi yang harus dipenuhi oleh data yang diambil.

- Comparison Operators: Operator yang digunakan untuk membandingkan nilai. Dalam konteks ini, operator `>=` digunakan untuk mendapatkan semua record dengan `bill_amount` lebih besar atau sama dengan 40. Contoh sintaks:

```
SELECT booking_id, bill_amount FROM orders WHERE  
    bill_amount >= 40;
```

Operator perbandingan digunakan untuk membandingkan dua nilai, seperti `=`, `>`, `<`, `>=`, dan `<=`.

2.9 Teknik Sorting

ORDER BY Clause: Klausa opsional yang digunakan untuk mengurutkan data dalam urutan tertentu (ascending atau descending). Contoh sintaks:

```
SELECT booking_id, bill_amount FROM orders WHERE bill_amount  
    >= 40 ORDER BY bill_amount ASC;
```

ORDER BY adalah klausa dalam SQL yang digunakan untuk mengurutkan hasil query berdasarkan satu atau lebih kolom.

2.10 Pengertian JOIN

JOIN adalah merupakan perintah SQL yang digunakan untuk menggabungkan data dari dua atau lebih tabel berdasarkan kolom yang sama.

2.11 Jenis-jenis JOIN

- LEFT JOIN: Mengambil semua data dari tabel kiri dan data yang cocok dari tabel kanan. Jika tidak ada kecocokan, hasilnya tetap menampilkan data dari tabel kiri.

- RIGHT JOIN: Mengambil semua data dari tabel kanan dan data yang cocok dari tabel kiri. Jika tidak ada kecocokan, hasilnya tetap menampilkan data dari tabel kanan.
- INNER JOIN: Mengambil hanya data yang cocok dari kedua tabel.
- OUTER JOIN: Mengambil semua data dari kedua tabel, dengan menampilkan NULL untuk data yang tidak cocok.

2.12 Proses Menggunakan INNER JOIN dengan Python

- Membuat Query: Query SQL dibuat sebagai string dalam Python. Contoh sintaks:

```
my_join_query = "SELECT menu_items.name, menu_items.  
price, menu.cuisine FROM menu_items INNER JOIN menu  
ON menu_items.item_id = menu.item_id"
```

- Menjalankan Query: Query dijalankan menggunakan metode execute dari objek cursor. Contoh:

```
cursor.execute(my_join_query)
```

- Mengambil Hasil: Hasil dari query dapat diambil menggunakan metode fetchall. Contoh:

```
results = cursor.fetchall()
```

3 Advanced Database Clients

3.1 Pengantar Fungsi MySQL

Fungsi MySQL adalah kode yang melakukan operasi tertentu dan mengembalikan hasil. Ini adalah blok kode yang dapat digunakan kembali untuk menyelesaikan tugas yang berulang.

3.2 Jenis-jenis Fungsi di MySQL

- String Functions: Digunakan untuk memanipulasi nilai string. Contoh:
CONCAT: Menggabungkan data dari dua tabel menjadi satu string. Sintaks: `SELECT CONCAT(column1, column2) FROM table_name;`
- Numeric Functions: Digunakan untuk melakukan operasi pada data numerik. Contoh:
AVG(): Menghitung rata-rata nilai. Sintaks: `SELECT AVG(column_name) FROM table_name;`
- Date and Time Functions: Digunakan untuk mengekstrak nilai tanggal dan waktu. Contoh:
NOW(): Mengambil waktu saat ini. Sintaks: `SELECT NOW();`
- Comparison Functions: Digunakan untuk membandingkan nilai dalam database. Contoh:
GREATEST(): Mengembalikan nilai terbesar dari daftar. Sintaks: `SELECT GREATEST(value1, value2);`
- Control Flow Functions: Digunakan untuk mengevaluasi kondisi dan menentukan jalur eksekusi. Contoh:
CASE: Menjalankan serangkaian kondisi dan mengembalikan nilai berdasarkan kondisi yang terpenuhi. Sintaks:

```
SELECT CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE result_default
END;
```

3.3 Penggunaan Fungsi dengan Python

Fungsi MySQL dapat diakses melalui Python untuk melakukan query dan mendapatkan hasil. Contoh penggunaan fungsi rata-rata untuk menghitung tagihan rata-rata pelanggan:

```
average_query = "SELECT AVG(bill_amount) FROM orders;"
cursor.execute(average_query)
results = cursor.fetchall()
```

3.4 Pengantar Fungsi Datetime

- Datetime: Kelas di Python yang memiliki beberapa fungsi bawaan untuk memformat dan mengubah variabel waktu dan tanggal.
- Import: Untuk menggunakan datetime, Anda perlu mengimpor kelas ini dengan sintaks:

```
import datetime as dt
```

3.5 Fungsi Utama dalam Datetime

- `datetime.now()`: Mengambil tanggal dan waktu saat ini.

```
current_time = dt.datetime.now()
print(current_time)
```

- `datetime.date`: Mengambil hanya bagian tanggal (hari, bulan, tahun).
- `datetime.time`: Mengambil hanya bagian waktu (jam, menit, detik).
- `timedelta`: Menghitung selisih antara dua nilai waktu atau tanggal.

```
from datetime import timedelta
week = dt.timedelta(days=7)
```

3.6 Contoh Sintaks untuk Mengupdate Slot Pemesanan

- Mengambil data dari tabel pemesanan:

```
cursor.execute("SELECT * FROM bookings")
```

- Menambahkan satu jam ke slot pemesanan:

```
new_booking_slot = booking_slot + dt.timedelta(hours=1)
```

- Mencetak hasil:

```
print(f"Booking ID: {booking_id}, Old Slot: {old_slot}, New Slot: {new_booking_slot}")
```

3.7 Pengantar Tentang Datetime

`datetime` adalah kelas di Python yang memiliki beberapa fungsi bawaan untuk memformat dan mengubah variabel waktu dan tanggal.

Anda dapat mengimpor `datetime` tanpa memerlukan PIP.

3.8 Fungsi-fungsi Dalam Datetime

- `datetime.now()`: Mengambil tanggal dan waktu saat ini.

```
current_time = dt.datetime.now()
print(current_time)
```

- `datetime.date`: Mengambil komponen tanggal (hari, bulan, tahun).
- `datetime.time`: Mengambil komponen waktu (jam, menit, detik).
- `timedelta`: Menghitung selisih antara dua nilai `datetime` dan dapat digunakan untuk menambah waktu.

```
from datetime import timedelta
week = timedelta(days=7)
```

3.9 Menggunakan Datetime untuk Mengatur Slot Pemesanan

Untuk menggeser setiap slot pemesanan satu jam ke depan, Anda perlu:

1. Mengimpor pustaka `datetime` dengan alias `dt`.
2. Menulis pernyataan SQL untuk mengambil semua data dari tabel pemesanan.
3. Menggunakan `timedelta` untuk menambah satu jam ke setiap slot pemesanan.
4. Mencetak nilai slot pemesanan yang baru.

Sintaks untuk mengupdate slot pemesanan:

```
new_booking_slot = booking_slot + dt.timedelta(hours=1)
print(f"Booking_ID: {booking_id}, Old_Slot: {booking_slot}, New_Slot: {new_booking_slot}")
```


3.10 Apa itu Stored Procedures

- **Stored Procedure:** Sebuah blok kode atau satu atau lebih query yang telah disiapkan sebelumnya dan dapat disimpan dalam database. Stored procedures dapat dipanggil sesuai kebutuhan, mirip dengan fungsi dalam pemrograman.
- **Perbedaan dengan Fungsi:** Fungsi hanya dapat memiliki parameter input, sedangkan stored procedure dapat memiliki parameter input dan output.

3.11 Langkah-langkah Menggunakan Stored Procedures

- **Membuat Stored Procedure:** Gunakan perintah `CREATE PROCEDURE` diikuti dengan nama procedure dan parameter yang diperlukan. Menganggil Stored Procedure:
- **Gunakan perintah CALL** diikuti dengan nama procedure. Menghapus Stored Procedure:
- **Gunakan perintah DROP PROCEDURE** diikuti dengan nama procedure, tanpa perlu menambahkan tanda kurung.

3.12 Manfaat Stored Procedures

- **Konsistensi Kode:** Kode yang sama digunakan setiap kali dipanggil, sehingga hasilnya dapat diprediksi.
- **Reusabilitas Kode:** Dapat digunakan berulang kali di berbagai tugas database.
- **Kemudahan Pemeliharaan:** Kode disimpan dalam satu blok yang dapat dipanggil, diedit, atau dihapus sesuai kebutuhan.

3.13 Sintaks untuk Membuat Stored Procedure

```
CREATE PROCEDURE procedure_name (parameters)
BEGIN
    -- logic of the procedure
END;
```

3.14 Mengakses Stored Procedure Menggunakan Python

Sintaks untuk mengeksekusi stored procedure:

```
CREATE PROCEDURE GetCustomersAndBillAmount()
BEGIN
    SELECT bookings.BookingID, CONCAT(bookings.
        GuestFirstName, bookings.GuestLastName) AS
        CustomerName, Orders.BillAmount
```

```

FROM bookings
INNER JOIN Orders ON bookings.BookingID = Orders.
    BookingID
WHERE BillAmount >= 50
ORDER BY BillAmount DESC;
END;

```

```

cursor.execute(stored_procedure_query)

```

3.14.1 Memanggil dan Mengambil Hasil dari Stored Procedures

`callproc()`: Metode untuk memanggil stored procedure.

Sintaks untuk memanggil stored procedure:

```

cursor.callproc("GetCustomersAndBillAmount")

```

`next()`: Fungsi untuk mengambil hasil berikutnya dari iterator hasil yang disimpan.

Sintaks untuk mengambil hasil:

```

results = next(cursor.stored_results())
dataset = results.fetchall()

```

Hasil dari stored procedure disimpan dalam bentuk list of tuples, di mana setiap tuple mewakili satu record.

```

for data in dataset:
    print(data)

```

```

CONCAT(bookings.GuestFirstName, ' ', bookings.GuestLastName) AS CustomerName,
Orders.BillAmount
FROM bookings
INNER JOIN
ORDERS ON bookings.BookingID=Orders.BookingID
WHERE BillAmount >=50
ORDER BY BillAmount DESC;
END
=====

In [7]: cursor.execute(stored_procedure_query)

In [8]: cursor.callproc("GetCustomersAndBillAmount")
Out[8]: ()

In [9]: results=next(cursor.stored_results())

In [10]: dataset=results.fetchall()

In [11]: for data in dataset:
          print(data)

(1, 'Anna Iversen', 86)

```

Figure 5:

3.15 Apa itu Database Connection Pooling?

- Database Connection Pooling: Merupakan teknik untuk menciptakan dan mengelola sekumpulan koneksi ke database, sehingga dapat meningkatkan efisiensi dan kecepatan akses database.
- Koneksi: Saluran komunikasi antara aplikasi dan database yang memungkinkan pertukaran data.

3.16 Cara Kerja Connection Pooling

- Contoh Visual: Bayangkan ada empat koneksi terbuka ke database MySQL. Dua koneksi sedang digunakan oleh klien, sementara dua lainnya siap digunakan oleh pengguna baru.
- Penugasan Koneksi: Ketika pengguna baru meminta akses, mereka akan diberikan salah satu koneksi yang tersedia. Setelah pengguna selesai, koneksi tersebut tidak ditutup, tetapi dikembalikan ke pool untuk digunakan kembali.

3.17 Mengatasi Keterbatasan Koneksi

- Multiple Pools: Jika semua koneksi sedang digunakan dan ada pengguna kelima yang ingin bergabung, solusi terbaik adalah membuat beberapa pool dengan jumlah koneksi tertentu di setiap pool. Ini memastikan selalu ada koneksi yang tersedia untuk pengguna baru.
- Pembuatan Koneksi Baru: Sistem dapat diprogram untuk membuat koneksi baru dalam pool jika tidak ada koneksi yang sesuai tersedia.

3.18 Keuntungan Connection Pooling

- Efisiensi Sumber Daya: Memanfaatkan sumber daya yang ada dengan lebih baik.
- Pengurangan Waktu: Mengurangi waktu dan usaha yang diperlukan untuk membangun koneksi baru.
- Sederhanakan Model Pemrograman: Mempermudah pengembangan aplikasi.
- Peningkatan Kinerja: Meningkatkan kinerja aplikasi Python saat terhubung ke database MySQL.

3.19 Database Connection Pool

- Definisi: Database connection pool adalah sekumpulan koneksi yang dikelola untuk memberikan akses yang aman dan terautentikasi ke database bagi banyak pengguna.

- Tujuan: Little Lemon ingin menyediakan koneksi yang efisien untuk tim pengembang web mereka melalui connection pooling.

3.20 MySQL Connection Pool Module

- Import Modul: Untuk menggunakan modul ini, Anda perlu mengimpor dengan sintaks:

```
from mysql.connector import pooling
```

- Fungsi Utama:
 - `pool_name`: Atribut kelas yang digunakan untuk mengidentifikasi nama pool yang akan digunakan. Jika tidak ditentukan, satu pool akan dihasilkan secara otomatis.
 - `pool_size`: Menentukan jumlah koneksi yang dibuat untuk pool. Defaultnya adalah lima, tetapi bisa dibuat hingga 32 koneksi.
 - `connection_ID`: ID unik yang diberikan untuk setiap koneksi dalam pool.

3.21 Metode Kelas dalam MySQL Connection Pool

- `get_connection()`: Meminta koneksi dari pool. Jika tidak ada koneksi yang tersedia, akan muncul error "pool exhausted".
- `is_connected()`: Fungsi Boolean yang mengembalikan nilai true atau false tergantung pada apakah koneksi berhasil dibuat.
- `close()`: Menginformasikan pool bahwa pengguna telah menyelesaikan sesi mereka, sehingga koneksi dapat dikembalikan ke pool untuk digunakan oleh pengguna baru.

3.22 Langkah-langkah Membuat Connection Pool

- Impor Modul:

```
from mysql.connector import pooling
```

- Buat Koneksi Pool:

```
pool = pooling.MySQLConnectionPool(pool_name="LittleLemonPool", pool_size=4, host='localhost', database='LittleLemon', user='username', password='password')
```

- Buat Daftar Pengguna: Buat list pengguna untuk koneksi pool.

- **Buat SQL Select Statement:** Buat pernyataan SQL yang menerima integer untuk ID yang berbeda.
- **Gunakan Loop:** Gunakan for loop dengan fungsi `range()` untuk mengulang hingga ukuran pool.
- **Cek Koneksi:** Tulis pernyataan untuk memeriksa apakah koneksi berhasil dibuat.
- **Instansiasi Cursor:** Buat cursor baru dari koneksi yang ada.
- **Tampilkan Informasi:** Gunakan `print()` untuk menampilkan informasi pengguna dan ID koneksi yang unik.
- **Fetch Data:** Gunakan metode `fetch_all()` untuk mengembalikan semua informasi yang sesuai dengan pernyataan SQL.
- **Tutup Koneksi:** Gunakan metode `close()` untuk mengembalikan koneksi ke pool setelah sesi selesai.

4 Working with a Database Client